MPOO 2027-1 · GRUPO 06

Parcial 1 · correcciones

Versión A del examen del 7 de octubre. Para cada pregunta vienen el enunciado, tu respuesta tal como la enviaste en el portal y su corrección. La calificación no va aquí: la ves con el profesor.

Antes de empezar. Este archivo lo subió el profesor a tu rama. Trae el cambio a tu computadora antes de tu siguiente push, o tu push va a fallar:

```
git checkout entregas_gutierrez_ixcmie
git pull --no-rebase
```

Qué tienes que hacer. En hojas, **a mano**, copia cada pregunta y escribe debajo tu respuesta corregida, usando la corrección que viene aquí. Donde la corrección ya te da la salida o el código, no basta con copiarlo: escribe **por qué** sale así, con la tabla de seguimiento de las variables, la causa del error o lo que devuelve tu método en cada renglón de la tabla. Debajo de esas preguntas viene qué escribir. Escanea las hojas o tómales foto, júntalas en un PDF y súbelo a esta misma carpeta, `entregas/gutierrez_ixcmie/examen/p1/`, con tu push de siempre.

Pregunta 0.1 · Preguntas abiertas

Pregunta

Contesta con tus palabras, en una a tres oraciones. Puedes apoyarte en un ejemplo pequeño de código
¿Qué diferencia hay entre una clase y un objeto? Usa la idea de plano e instancia.

Tu respuesta

```
La clase contiene a los atributos que describen al objeto, por lo que la diferencia recae en
que la clase es una estructura base (tipo una plantilla) que sirve para conocer y operar con
los datos más abstractos e importantes de un objeto siendo así la clase el conjunto de
atributos de un objeto, mientras que el objeto es una variable completa único (que puede
clasificarse).
```

Corrección. Bien que presentes la clase como una plantilla. Pero dices que el objeto es "una variable", y eso es un error conceptual: la variable solo guarda la referencia. El objeto es una instancia concreta de ese plano con sus propios valores de los atributos, y la clase define atributos y también métodos.

Pregunta 0.2 · Preguntas abiertas

Pregunta

¿Qué es la firma de un método y qué partes la forman? Explica si incluye el tipo de retorno.

Tu respuesta

```
La estructura para declarar/formar uno, la conforman public, static, el tipo de dato que
retorna (si es que retorna uno), el nombre del método, y sus parámetros.
```

Corrección. Bien que incluyas el nombre y los parámetros. Pero pones en la firma public, static y el tipo de retorno, y no van ahí. La firma es solo el nombre más los tipos de los parámetros en orden; el retorno, los modificadores y los nombres de los parámetros quedan fuera.

Pregunta 0.3 · Preguntas abiertas

Pregunta

Compara la anatomía de una variable local y un atributo de instancia: dónde se declaran, valor inicial y cuánto viven.

Tu respuesta

```
Variable Local:
–Se declaran al usarse en un método
–Valor inicial = 0
–Viven hasta que el método termine de usarse
Atributo de instancia:
–Se declaran durante el desarrollo de una clase
–Viven mientras la clase esté en uso
```

Corrección. Bien: la local se declara en el método y vive mientras este se ejecuta, y el atributo se declara en la clase. Pero la local no empieza en 0: hay que asignarle un valor antes de leerla. El atributo sí recibe un valor por defecto y vive lo que vive el objeto, no la clase.

Pregunta 0.4 · Preguntas abiertas

Pregunta

¿Qué significa null en una referencia? Explica qué ocurre al llamar un método sobre esa referencia.

Tu respuesta

Sin respuesta.

Corrección. Sin respuesta.

Pregunta 0.5 · Preguntas abiertas

Pregunta

¿Cuándo conviene usar for-each para recorrer un arreglo y cuándo necesitas un recorrido con índice?

Tu respuesta

```
Dependerá de si lo que interesa conocer es el contenido del índice o el índice (la posición)
por si misma, for each no toma en cuenta la posición sino el contenido. El recorrido con un
índice es acertado manejarlo con for.
```

Corrección. Correcto: for-each recorre los valores sin usar la posición, y cuando necesitas la posición (para consultarla o para sustituir un elemento) usas un for con índice.

R1 · ¿Qué imprime?

Pregunta

Escribe la salida exacta, una línea por renglón. El fragmento está en un main válido.

```
StringBuilder ruta = new StringBuilder("S");
StringBuilder alias = ruta;
for (int i = 1; i <= 3; i++) {
    alias.append(i / 2);
    System.out.println(ruta);
}
String texto = "S" + 2 + 1;
System.out.println(texto);
double mitad = (double) (7 / 2);
System.out.println(mitad);
```

Tu respuesta

Sin respuesta.

Corrección. Sin respuesta. La salida correcta era S0 / S01 / S011 / S21 / 3.0: alias y ruta son el mismo StringBuilder, 1/2, 2/2 y 3/2 agregan 0, 1 y 1, la concatenación va de izquierda a derecha y 7/2 da 3 antes del cast.

Qué escribir a mano. La salida ya la tienes en la corrección. Escribe por qué sale así: una tabla de seguimiento con el valor de cada variable en cada vuelta y en cada println.

R2 a) · ¿Compila, truena o funciona?

Pregunta

Cada inciso va en un main independiente. Indica no compila, compila pero truena o funciona y da la causa en una frase. Si funciona, escribe la salida (separa líneas con /).

```
double medida = 6.8;
int unidades = medida;
System.out.println(unidades);
```

Tu respuesta

Clasificación

```
No compila
```

Causa o salida

```
Se asigna unidades (en int) a medida, previamente asignada con un número double. El error se marca al no ser el mismo tipo de dato.
```

Corrección. Es correcto que no compila. La causa va bien encaminada al señalar el choque entre double e int, pero 'no ser el mismo tipo de dato' no basta, porque Java sí acepta pasar de int a double. Lo que falla es que no hay conversión implícita de double a int (se perdería información), así que hace falta un cast explícito.

Qué escribir a mano. La clasificación ya la tienes en la corrección. Escribe por qué: qué línea causa el error y qué regla de Java lo provoca o, si funciona, cómo se llega a esa salida.

R2 b) · ¿Compila, truena o funciona?

Pregunta

Cada inciso va en un main independiente. Indica no compila, compila pero truena o funciona y da la causa en una frase. Si funciona, escribe la salida (separa líneas con /).

```
int[] canales = {2, 5, 8};
int indice = canales.length;
System.out.println(canales[indice]);
```

Tu respuesta

Clasificación

Sin respuesta.

Causa o salida

```
2 5 8
```

Corrección. No marcaste la clasificación y escribiste "2 5 8" como salida. Compila pero truena: length vale 3 y el último índice válido es 2, así que canales[3] lanza una excepción de índice fuera de rango y no se imprime nada.

Qué escribir a mano. La clasificación ya la tienes en la corrección. Escribe por qué: qué línea causa el error y qué regla de Java lo provoca o, si funciona, cómo se llega a esa salida.

R2 c) · ¿Compila, truena o funciona?

Pregunta

Cada inciso va en un main independiente. Indica no compila, compila pero truena o funciona y da la causa en una frase. Si funciona, escribe la salida (separa líneas con /).

```
String codigo = new String("F2");
System.out.println(codigo == "F2");
System.out.println(codigo.equals("F2"));
```

Tu respuesta

Clasificación

Sin respuesta.

Causa o salida

```
F2
```

Corrección. No marcaste la clasificación y "F2" no es la salida. Funciona e imprime false / true: == compara si son el mismo objeto (new String crea otro) y equals compara el contenido.

Qué escribir a mano. La clasificación ya la tienes en la corrección. Escribe por qué: qué línea causa el error y qué regla de Java lo provoca o, si funciona, cómo se llega a esa salida.

R3 · Modelado: base, reto y justificación

Pregunta

Cada multímetro conserva su inventario y nace sin responsable. Se cuenta el total de prestados. Escribe prestar(persona) y una consulta del responsable. Supón que solo se llama prestar con un aparato disponible y un nombre válido.

Base. Declara al menos dos atributos de instancia con tipo, uno static y una constante final (¿también static?). Completa constructor y dos métodos: cambio y consulta.

Reto. ¿La cuenta de prestados pertenece a cada aparato o a todos? ¿Qué dato nunca cambia?

Justifica. Una línea por decisión: tipo, instancia/clase, final o mutable, constructor y métodos.

Cuerpos de 1 a 3 líneas, sin estructuras de control ni main. Hay más de un modelo correcto. Un elemento sin justificar vale la mitad.

Tu respuesta

Clase

Sin respuesta.

Justificaciones

Sin respuesta.

> **Corrección.** Sin respuesta. Se esperaba algo como: final String inventario y String responsable como atributos de instancia; static int prestados como cuenta compartida; un constructor que use this; prestar(persona), que asigna el responsable y suma uno a prestados; y una consulta que devuelva el responsable.

R4 · Código con repetición

Pregunta

Un equipo guarda sus folios como enteros no negativos. Devuelve cuántos dígitos tiene folio usando while y división entre 10. El folio 0 tiene un dígito. No conviertas el número a texto.

```
static int contarDigitos(int folio)
```

Ejemplos (argumentos → retorno):

4072 → 4 · 0 → 1 · 8 → 1

Escribe el método completo. No imprimas ni leas del teclado.

Tu respuesta

```
public static int CuantosDigitos(int[]folio){
    int Digi2 = 0;
        for(int i=0;i<folio.length;i++){
            Digi2 ++;
        }
}
```

> **Corrección.** Recibes un arreglo int[] en lugar de un int, el nombre no es contarDigitos, usas for en lugar de while, falta el return y en ningún momento divides entre 10, así que cuentas casillas del arreglo y no dígitos. Lo correcto era: si folio es 0, devolver 1; si no, mientras folio > 0 hacer folio /= 10 y sumar uno al contador, y al final devolverlo.

Qué escribir a mano. El método corregido completo y, para cada ejemplo o renglón de la tabla del enunciado, qué devuelve tu método y por qué.

R5 · Código con selección

Pregunta

Clasifica la carga de un equipo. Usa switch clásico sobre carga / 10. Puedes validar antes con if. Devuelve el texto de la tabla. Los límites están incluidos. No imprimas ni leas del teclado.

```
static String nivelCarga(int carga)
```

Regla o valor recibido	Texto devuelto
Menor que 0 o mayor que 50	"INVALIDA"
0 a 19	"BAJA"
20 a 39	"MEDIA"
40 a 50	"ALTA"

Tu respuesta

```
static String nivelCarga(int R);
public switch(R){
    case 1: if(R>= 0 && R<=19){
                System.out.println("BAJA");
             }
    case 2: if(R>= 20 && R<=39){
                System.out.println("MEDIA");
             }
    case 3: if(R>= 40 && R<=50){
                System.out.println("ALTA");
             }
    case 4: if(R< 0 && R>50){
                System.out.println("INVALIDA");
             }
    default: System.out.println("NULL");
}
```

Corrección. La cabecera está bien y los rangos de los if (0-19, 20-39, 40-50, con 20, 40 y 50 incluidos) coinciden con la tabla. Pero el switch va sobre R y no sobre carga / 10, así que cada case solo atrapa un valor literal, y sin break se cae al siguiente. La condición R<0 && R>50 nunca se cumple (iba ||), imprimes en lugar de devolver y no hay ningún return. Lo esperado era validar con if (carga < 0 || carga > 50) return "INVALIDA"; y luego switch (carga / 10) con case 0: case 1: return "BAJA"; case 2: case 3: return "MEDIA"; default: return "ALTA".

Qué escribir a mano. El método corregido completo y, para cada ejemplo o renglón de la tabla del enunciado, qué devuelve tu método y por qué.